

riverrun

Post-Quantum Behavioral Privacy on Solana,
and a Ruler for What It Is Actually Worth

Manuel Guilherme Galmanus

Independent Researcher

m.galmanus@gmail.com

July 2026

Abstract

On a public ledger, hiding *who* performed an action is a different problem from hiding *how much* money moved, and a harder one to reason about honestly. This paper describes **riverrun**, a system for behavioral privacy on Solana: a member commits an intent, and later executes exactly that intent without anyone being able to say which member did it. The proof is a transparent, post-quantum STARK that binds membership, a per-round nullifier, and the public action into one object, with no trusted setup. A synchronized round of k members collapses into a single proof and a single verification.

The paper’s second half is a warning aimed at riverrun and at every system like it. Every anonymity pool advertises a number, $1/k$, that counts members. That number is a fiction, because it ignores where the members’ money came from, and the funding graph is public. We give a ruler—the effective anonymity-set size of Serjantov and Danezis, conditioned on funding provenance—and run it against a live pool on Solana mainnet. A set advertising $k = 30$ delivers an effective 6.5, and one member has an anonymity of exactly 1. The same ruler, turned into a pre-flight check, tells a user the anonymity *they* would get in any pool before they act. Every claim of the form “ X is bound” in the implementation has a test that was verified to fail without the constraint it tests; three such tests turned out to be decorative, and we say which. Where the system does not yet hold—an unaudited hand-rolled circuit, off-chain verification, example-grade security parameters—we say so plainly.

Contents

1	The problem, in one picture	1
2	The idea: commit an intent, execute it unlinkably	2
3	The proof: one object, three bindings	3
3.1	What the circuit proves	3
3.2	Knowing the weld holds	4
3.3	A leak we found by measuring	5
4	The synchronized round: one crowd, one proof	5
5	The ricorso: the crowd is reborn	6

6	The ruler: your k is not your k	7
6.1	The advertised number counts the wrong thing	7
6.2	The formula, and why its direction matters	7
6.3	The measurement	8
6.4	The same ruler, turned on riverrun—and on the user	8
7	Cost, and the on-chain reality	9
7.1	What one execution costs	9
7.2	Verifying on-chain: measured, and honest about the wall	10
7.3	What the chain enforces today	10
8	What does not hold yet	11
9	The threat model, stated plainly	11
10	Related work	12
11	Conclusion	12
A	Reproducing every number in this paper	13

1 The problem, in one picture

Imagine a town where every letter anyone ever mailed is pinned to a public board, forever. Not the contents—those are sealed—but the envelopes: who sent what to whom, when, and how heavy the package was. You would learn a great deal about that town without opening a single letter. You would know who pays whom, who is suddenly buying, who moves money at 3 a.m., who follows whose lead an hour later.

A public blockchain is that board. People imagine that a wallet address, being a random string rather than a name, keeps them private. The opposite is closer to the truth. The address is a stable handle, and everything done under it accumulates into a pattern—a *behavioral fingerprint*. When you act, how much, which protocols you touch, in what order: this is enough for modern chain-analysis to cluster your wallets, attach a real identity, and, increasingly, to predict and front-run what you are about to do. The tools that do this get better every month, because the data is permanent and public and the models feeding on it are hungry.

There are two ways to want privacy here, and they are not the same. One is to hide *the money*: the amounts, the balances. The other is to hide *the behavior*: which person, out of a crowd, performed a public action. Most privacy work on blockchains has been about the money—confidential transfers, shielded balances, mixers that break the link between a deposit and a withdrawal of value. This paper is about the second kind. Amounts are not hidden here and are not the point. What we sever is the link between an *actor* and an *action*: the observer sees that a swap happened, that a vote was cast, that a protocol was used, but cannot say who did it.

Richard Feynman liked to say that if you cannot explain something simply, you do not understand it. So here is the whole idea in one sentence, and the rest of the paper is just this sentence made

precise and made real: *you disappear into a crowd of people who all look like they could have done the thing you did.*

The name is borrowed. *riverrun* is the first word of James Joyce’s *Finnegans Wake*, a book with no beginning—its last sentence flows into its first, a river returning to its own source. That image turns out to be the exact shape of the strongest attack on systems like this, and of the defense against it, so we kept the name and, in Section 6, made the circle do real work.

2 The idea: commit an intent, execute it unlinkably

Tornado Cash, the well-known Ethereum mixer, breaks the link between putting money *in* and taking money *out*. You deposit one coin into a pool; later, from a fresh address, you withdraw one coin, proving in zero knowledge that you are entitled to a withdrawal without revealing which deposit was yours. The pool is a crowd, and you are hidden in it.

riverrun keeps the shape of that trick but changes the payload. Instead of a coin, the thing hidden is a *behavior*. The flow has three steps.

Commit. A member publishes a commitment $\ell = \text{Rescue}(s, a)$ into a public set, where s is a secret only they know and a is the action they intend to perform—“swap 1 SOL for USDC on venue V ”, say, encoded as a hash. Publishing ℓ announces, to everyone, that *somebody in this set intends action a* , while revealing nothing about who. The set is a Merkle tree; its root is the public fingerprint of the whole crowd.

Execute. Later—ideally in a synchronized round where many members perform the same action at the same moment—the member performs a from a fresh identity, accompanied by a zero-knowledge proof of a single sentence:

“I know a secret s whose commitment $\text{Rescue}(s, a)$ is a leaf under the public root R , and my nullifier for this round is $n = \text{Rescue}(s, r)$.”

The action a is public—the observer sees it happen. The author is not.

Settle. The pool verifies the proof, checks that the nullifier n has not been seen this round, records it as spent, and releases the action. The public transcript is just $\{R, a, n\}$, with no link back to any commitment.

Two questions decide whether this is privacy or theater, and both are about the nullifier. First, why have one at all? Because without it, a member could execute their action a thousand times from a thousand fresh identities, and the “crowd” would be one person wearing a thousand masks. The nullifier is a token, derived from the secret and the round, that can be spent exactly once per round. It is *unlinkable*: $\text{Rescue}(s, r)$ reveals nothing about s , and the nullifiers a single member produces in different rounds, $\text{Rescue}(s, r_1)$ and $\text{Rescue}(s, r_2)$, cannot be tied to each other. So it enforces “one action per member per round” without ever identifying the member. This is a known and well-studied primitive; PLUME [6] formalizes exactly this kind of deterministic nullifier and proves its uniqueness and secrecy.

Second, and this is the subtle one: how do we know the nullifier n that a member reveals really came from the *same* secret that proved membership? If those two facts—“I am a member” and “my nullifier is n ”—are proven separately, a member could prove membership honestly and then present a nullifier belonging to somebody else, spending a stranger’s turn or hiding that they

have already spent their own. The two halves must be welded together inside one proof. Making that weld correct, and knowing it is correct, is the technical heart of the next section.

3 The proof: one object, three bindings

A zero-knowledge proof lets you convince someone that a statement is true while revealing nothing beyond its truth. There are two broad families in use on blockchains today, and the choice between them is a genuine fork with real costs on each side.

The first, pairing-based SNARKs such as Groth16, produce tiny proofs—a couple of hundred bytes—that verify cheaply. Their price is a *trusted setup*: before anyone can prove anything, a one-time ceremony must generate a structured “proving key”, and if the secret randomness of that ceremony is ever recovered, the whole system can be forged. For a circuit as large as a Merkle-path check, that key is tens of megabytes that must be produced, trusted, and shipped to every prover. And the security rests on the hardness of discrete logarithms over elliptic curves—which a large enough quantum computer breaks.

The second family, STARKs, are built from nothing but a hash function. No pairings, no elliptic curves, no ceremony, no per-circuit setup. Their security rests on the collision-resistance of a hash, which is believed to survive a quantum adversary. The price is proof size: a STARK is kilobytes, not bytes. *riverrun* takes this side of the trade, deliberately, and Section 7 measures exactly what it costs and what it buys.

3.1 What the circuit proves

Concretely, *riverrun* uses a STARK over a Rescue-Prime [12] Merkle tree—an “arithmetization-oriented” hash chosen because its structure is cheap to express as the polynomial constraints a STARK understands. The proof’s public inputs are the four-tuple

$$\{ R, n, r, a \},$$

the root, the nullifier, the round, and the action. Its private inputs—the witness, which the proof hides—are the secret s , the leaf’s position in the tree, and the Merkle path. A single secret must satisfy all of the following at once:

1. **Membership.** $\text{Rescue}(s, a)$ is a leaf under the public root R , via the private path. (The member is genuinely in the set.)
2. **Nullifier binding.** $n = \text{Rescue}(s, r)$: the revealed nullifier is this member’s, for this round.
3. **Action binding.** The very leaf that resolves to R commits to the *public* action a . (The member executes the intent they registered, not a different one.)

Because all three read from the same private s , no member can mix and match: prove membership as Alice while spending Bob’s nullifier, or prove they committed action a while executing action a' . That is the weld.

What a STARK actually is, in one paragraph. Write your computation as a big table—rows are time-steps, columns are registers—so that “the computation was done correctly” becomes “every adjacent pair of rows obeys a fixed rule”. Turn each column into a polynomial that passes through its values. The rules become equations these polynomials must satisfy. Now the prover commits to the polynomials (with a Merkle tree of their values on a larger domain) and the verifier challenges them at random points; a cheat would have to make a *wrong* low-degree polynomial agree with a right one almost everywhere, which

is impossible unless they are equal. FRI is the machinery that lets the verifier check “this really is a low-degree polynomial” by repeatedly folding it in half and spot-checking. The disanalogy with a normal exam: the verifier never sees the answers, only a few random cells and a proof that all the cells are consistent— which is why the witness stays hidden.

In riverrun’s case the table has width nine (Figure 1): six columns hold the Rescue hash state, one holds a Merkle direction bit, and two—the *carry*—hold the secret across the one hash cycle that computes the nullifier before the Merkle path begins. A single constraint at the first row, the “start-tie”, forces the nullifier’s input to equal the value the path will hash. Delete that one constraint and the weld comes apart.

col	Rescue state 0...5	bit 6	carry 7, 8
cycle 0	hash $(s_0, s_1, r) \rightarrow$ nullifier n	—	$s_0 \quad s_1$
row 7 (load)	$s_0, s_1 \quad a_0, a_1 \quad 0$	—	(cleared)
cycles 1.. $d+1$	Merkle path up to root R	path bit	—

Figure 1: The width-9 execution trace. Cycle 0 computes the nullifier from the secret and the round; row 7 loads the leaf preimage—the same secret, plus the public action—so the leaf commits to both; the remaining cycles walk the Merkle path to the public root. The start-tie at row 0 forces the nullifier’s input and the carried secret to be equal, welding the two halves. The carry is confined to cycle 0 on purpose: a column held constant would leak (Section 3.3).

3.2 Knowing the weld holds

A privacy proof that is subtly wrong is worse than no proof, because it advertises a guarantee it does not deliver. So the interesting question is not “did we write the constraints” but “how do we know they bite”. The answer used throughout riverrun is *mutation testing*: for every claim of the form “ X is bound”, there is a test, and we delete the constraint that is supposed to enforce X and confirm the test then fails. If deleting the constraint changes nothing, the test was not testing the constraint—it was passing for some other reason—and it is worthless.

This is not a formality. Three tests written during development turned out to be exactly that kind of worthless, and mutation testing is how they were caught.

- The first tests for the *action* binding checked that announcing a different public action was rejected. They passed even with the binding constraint deleted—because the action feeds the proof’s Fiat–Shamir transcript, so a mismatched public input is rejected anyway, for a reason having nothing to do with the constraint under test. The real attack, which the useful test performs, is to hash the committed action into the leaf (so the Merkle path still resolves) while *announcing* a different one. With the constraint deleted, that forged proof verifies. With it, nothing usable comes out.
- The nullifier-binding negative tests that only varied public inputs passed on the boundary assertions alone, with no start-tie at all. The test that actually guards the weld builds the trace an attacker would want—hash secret A , carry member B into the path—and only the start-tie stops it.
- In the *ricorso* extension (Section 5) a whole check—that the revealed migration nullifier comes from the migrating secret—had *no* test at all until a mutation left every test green and exposed the gap.

None of this makes the circuit audited. It is hand-rolled, and a passing negative test is necessary but not sufficient for soundness; an independent review is on the roadmap and is named as an

open item, not glossed. What mutation testing buys is the removal of a specific, common, and embarrassing failure: tests that look like they prove something and prove nothing.

3.3 A leak we found by measuring

One more story, because it is the kind of thing that separates a system that was measured from one that was merely asserted. The first version of the circuit carried the secret in two “carry” columns held *constant* across the whole execution trace. This looked harmless. It was not. A STARK proof works by opening the prover’s committed data at random points, and a column that is constant has a constant low-degree extension—so the secret appeared, verbatim, in essentially every opening. We only knew because we wrote a test that checked, over many proofs, whether the raw bytes of the secret appeared in the proof. They did: twenty times out of twenty. Confining the carry to the eight rows where it is actually needed fixed it: zero out of twenty. A single-sample test would have missed it. This is also why we are careful to call the result “the witness is not on the wire” and not “zero-knowledge”—the underlying prover does not randomize the witness, so formal zero-knowledge is a claim we decline to make.

4 The synchronized round: one crowd, one proof

The privacy argument rests on a crowd. If k members perform the same action in the same round, an observer’s chance of attributing any one execution to its author is at best $1/k$. The natural way to run this is a *synchronized round*: many members acting identically at one moment, so that there is no timing, no ordering, nothing to separate them.

Here is a small observation with a large payoff. That synchronized round is exactly the structure that lets many proofs collapse into one. A STARK proof is a statement about an execution trace—a long table of numbers. Proving k members separately means k traces, k proofs, and, on-chain, k verifications. But the k sub-traces are independent, so they can be laid end to end into one longer trace, with the periodic constraints that fire “once per member” repeating along it, and a *seam* constraint that switches off the linking logic at each boundary so one member’s last row does not bleed into the next member’s first. The result is one proof for the whole round, and one verification.

The savings are not marginal. Table 1 gives the measured numbers.

members k	one batched proof	k separate proofs	saving
4	19,772 B	45,300 B	2.3×
64	43,684 B	1,074,169 B	24.6×

Table 1: Batching a synchronized round. At $k = 64$ a per-member scheme ships over a megabyte of proof and needs 64 verifications; the batched round is one proof and one verification.

Honesty requires two caveats, both structural. First, the prover of a batched round must know every member’s secret. For a crowd of mutually distrustful strangers that is not privacy—it is a custodian, and batching there would need distributed proving. But for a single operator running k identities—an agent fleet, a market maker with many wallets, precisely an audience this problem serves—one prover holds all k secrets already, and the 24.6×

Second, batching trades prover time for proof size: the batched trace is k times longer, and proving is super-linear, so the round costs more wall-clock to prove than the sum of the individual proofs. It is the right trade when what you are paying for is on-chain verifications, which is the scarce resource.

5 The ricorso: the crowd is reborn

Now the name earns its keep. *Finnegans Wake* is built on Giambattista Vico’s cycle of history: ages that repeat, and a fourth age, the *ricorso*, the return that begins the whole thing again. riverrun already borrows the book’s circularity once, for the funding graph in Section 6. The ricorso is the other borrowing, and it closes a leak that the basic construction has and that is easy to miss.

Here is the leak. A member’s per-round nullifier changes every round, which unlinks one execution from another. But their *leaf*— $\text{Rescue}(s, a)$ —never changes. It is published once at commit and stays in the tree forever. Two consequences follow, neither of them stated in the system as described so far. Anyone who ever learns your leaf, through a bug or a subpoena or a careless client, can link you across every round you have ever acted in, past and future—the per-round nullifiers protect executions from each other, but nothing protects a member from their own history. And a set that only ever grows is itself a record: join-order is public, so the set is a timeline of who arrived when.

The ricorso fixes both by letting the set be *reborn* each cycle. A member holds a fresh secret per cycle, derived from one master secret:

$$s_c = \text{Rescue}(s, c, \text{dom}_{\text{cycle}}), \quad \ell_c = \text{Rescue}(s_c, a),$$

and at each rebirth publishes the new leaf ℓ_c while proving, in zero knowledge, that it descends from *some* leaf under the previous cycle’s root—without revealing which. A migration nullifier $\text{Rescue}(s, c, \text{dom}_{\text{migrate}})$, spent once per cycle, stops one seat from becoming several at the rebirth. The two domain tags must differ; if they did not, publishing a migration nullifier would leak the next cycle’s secret, and with it the member’s next leaf, making the rebirth public and pointless—a requirement the implementation pins with a test. History stops accumulating, because the leaves an observer can see this cycle are unlinkable to the leaves they saw last cycle.

In the current implementation this relation is specified and checked “in the clear”—a function that takes the witness and returns whether the migration is valid, with mutation-tested coverage of each of its four checks—but the STARK that proves it *without* the witness is designed and not yet built. The reason is a concrete and slightly annoying interaction: the migration circuit needs five hash cycles before the Merkle path, which shifts the valid anonymity-set sizes to $\{8, 2048\}$, while the execution circuit needs sizes $\{4, 64, 16384\}$, and the two do not intersect, so shipping the migration proof also means padding the execution circuit to realign the two. That is a day of careful work on the most delicate code in the system, and the honest place to stop, with the execution path green and tested, is at the specification—exactly where the nullifier binding itself stopped before it was later built and mutation-verified.

6 The ruler: your k is not your k

Everything so far has been about building the pool. This section is about measuring it, and it applies not just to riverrun but to every anonymity pool of this kind. It is, we think, the most useful part of the paper, because it names a number that is quietly wrong across an entire category of systems.

6.1 The advertised number counts the wrong thing

Every pool reports its privacy as $1/k$: there are k members, so an observer has a one-in- k chance of guessing the actor. That counts *members*. It assumes the members are interchangeable—that knowing anything about one tells you nothing about which one acted.

They are not interchangeable, because of where their money came from. Every member had to fund their wallet somehow, and on a public ledger the funding trail is public. Trace a member’s wallet backward and you often reach an attributable origin: a major exchange’s withdrawal address, a known service, a doxxed funder. Now sort the whole set by these origins. If the acting identity’s origin is observable too—and a fresh withdrawal wallet has to be funded from *somewhere*—then learning the actor’s origin does not leave k suspects. It leaves only the members who share that origin. The crowd you are actually hidden in is your *provenance class*, not the whole set.

This is not a new idea in the abstract. Serjantov and Danezis [1] showed in 2002 that the honest measure of anonymity is not the size of the set but the *entropy* of the distribution linking suspects to the event—the “effective anonymity-set size”. And measuring the gap between a mixer’s advertised and true anonymity is an established line of work on Ethereum [2, 3, 4, 5]. Two things are new here: the channel (funding provenance, which those works do not condition on—they use address reuse, timing, wallet fingerprints) and the chain (none of it has been run on Solana).

6.2 The formula, and why its direction matters

Partition the k members into provenance classes of sizes $n_1, n_2, \dots$. The adversary learns the actor’s class c with probability n_c/k , and is then left with a uniform guess among that class’s n_c members. The residual uncertainty is the average, over which class the actor came from, of the log of the class size:

$$H = \sum_c \frac{n_c}{k} \log_2 n_c, \quad k_{\text{eff}} = 2^H. \quad (1)$$

k_{eff} is the size of the crowd the member is genuinely hidden in. One class holding everyone gives back $\log_2 k$ —nothing was partitioned, the set is worth its advertised size. All-singleton classes give $H = 0$, an effective k of one: every member individually identified.

The formula, worked by hand. Take a pool of ten members. Suppose the funding trace sorts them into three provenance classes: six came from exchange A, three from exchange B, and one from a wallet all its own. The advertised anonymity is $1/10$. But condition on the actor’s origin. With probability $6/10$ the actor is an “A” member, and then the adversary cannot tell which of the six—that is $\log_2 6 \approx 2.58$ bits. With probability $3/10$ it is a “B” member, $\log_2 3 \approx 1.58$ bits. With probability $1/10$ it is the lone member, and the adversary knows exactly who: $\log_2 1 = 0$ bits. Average them:

$$H = \frac{6}{10}(2.58) + \frac{3}{10}(1.58) + \frac{1}{10}(0) = 2.02 \text{ bits}, \quad k_{\text{eff}} = 2^{2.02} \approx 4.1.$$

The pool sold ten and delivered four. And the lone member has an anonymity of exactly one: for them the pool is a mirror with their name on it, no matter how big the crowd behind them.

The direction of this formula is the whole thing, and the first implementation had it backwards. It is tempting to measure how *concentrated* the classes are—the min-entropy of the class distribution—but that reports catastrophe precisely when the measurement found *nothing* to partition on. A single class of seven, which is the happy case (no origins found, everyone alike), came out as “effective $k = 1$ ” under the wrong formula, when the honest reading is “effective $k = 7$ ”. Following Serjantov and Danezis, and against the normalized variant of Díaz et al. (which scores a set of one as a perfect 1.0), we use the residual entropy of Equation 1 directly. Four unit tests pin the direction, and one of them caught the arithmetic a second time.

6.3 The measurement

We ran this against a live Tornado-style SOL privacy pool on Solana mainnet, sampling 30 real depositors and tracing each one’s funding graph backward with a bounded, public-RPC walk. Table 2 is the result.

depositors sampled	30
reach an attributable origin	11/30 (37%), mean 1.18 hops
provenance classes	12
advertised k	30
effective k	6.5
worst case	1

Table 2: A live mainnet anonymity set, measured. A set advertising 30 delivers an effective 6.5. At least one depositor sits alone in their provenance class: for that person the pool provides no anonymity at all against an adversary who reads the funding graph, while the pool’s own dashboard would report their anonymity as one-in-thirty.

Three honesties about this number. It is not a flaw in that particular pool: no deposit-pool design can control where its users’ money came from, which is exactly why the channel goes unmeasured and keeps working as an attack. It is a *floor*, not a verdict: a public RPC following only SOL transfers to bounded depth sees less than a funded analyst with a maintained tag database and token-flow tracing, so the true leak is at least this large. And 37% is reassuringly in the same range—20 to 35%—that the Ethereum literature [4] finds with behavioral heuristics on Tornado Cash, which is a sanity check on the tool: a result of 95% would have been a reason to distrust the instrument, not the pool.

6.4 The same ruler, turned on riverrun—and on the user

Measuring other people’s pools and not your own is marketing. The same function runs over riverrun’s own constructions. Against a synthetic population of 2000 members, the “rooted decoy” construction that the field ships is worth an effective 500; the circular, origin-free construction—the funding cycle with no beginning, the *Finnegans Wake* image made numeric (Figure 2)—is worth the full 2000.

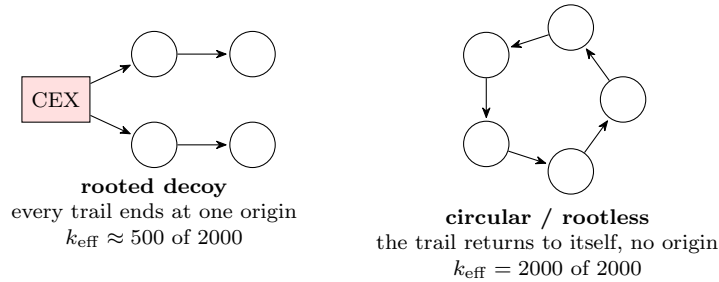


Figure 2: Two ways to fund a crowd. On the left, the construction the field ships: every wallet’s funding traces back to one attributable origin, so an adversary following the graph collapses the crowd. On the right, the *riverrun* construction: the funding trail is a cycle with no beginning—Vico’s *ricorso*, the river that returns to its source—so there is no origin to name. The same 2000 members are worth a quarter of their advertised size on the left and their full size on the right.

That is the circularity defense stated as a number instead of a metaphor, with the honest caveat

the tool prints: a cycle with no attributable origin holds only while its funding sources are themselves unattributable; one known exchange entry degrades it from “no origin” to “ambiguous origin”, which is still a large win and a smaller, measured one.

The most useful turn of the ruler, though, is toward the user. The measurement above audits a pool after the fact, which helps auditors. Flip it: before a user deposits into *any* pool, trace *their* wallet’s funding graph, sample the pool’s current depositors, and tell them the anonymity *they personally* would get there. The tool is protocol-agnostic—it reads public chain data, not any pool’s internals—so it works against a program it has never seen. It returns one of three verdicts, each with an action:

- **Exposed:** you would be alone in your provenance class. The pool’s size is irrelevant to you; fund a fresh wallet from a source other depositors also use, or wait for a same-origin crowd.
- **Weak:** only a small minority shares your class; your real anonymity is far below the headline.
- **OK:** a healthy fraction shares your class, so the crowd is genuinely yours—within the trace bound, meaning “no cheap attribution found”, never “anonymous”.

This is where the leverage is. Auditing pools reaches the handful of people who audit pools. Protecting a user at the moment they act reaches every user of every privacy protocol on the chain.

7 Cost, and the on-chain reality

A privacy tool nobody can afford to run is a paper, not a tool. So we measured.

7.1 What one execution costs

Table 3 gives the prover and verifier cost of a single execution, across anonymity-set sizes.

set size	proof	prove	verify	setup artifacts
4	11,929 B	2.0 ms	0.20 ms	0 bytes
64	17,045 B	3.7 ms	0.35 ms	0 bytes
16,384	19,064 B	8.1 ms	0.43 ms	0 bytes

Table 3: One execution, measured. An anonymity set of 16,384 members proves in 8 milliseconds. Proof size grows logarithmically: $4096\times$ the members costs $1.6\times$ the proof.

The column that decides whether any of this is deployable is the last one. A pairing-based prover needs a proving key produced by a ceremony and shipped to every client—for a Merkle circuit of this depth, tens of megabytes that must exist, be trusted, and be distributed before anyone proves anything. *riverrun*’s prover is the code; there is nothing to distribute and nothing to trust. For the audience this serves—agents, market makers, ordinary users proving before each action—that is the difference between a tool that ships and one that needs an install.

7.2 Verifying on-chain: measured, and honest about the wall

The hard part is on-chain verification. If the chain itself checks the proof, no one has to be trusted; if it does not, someone does. So we built a Solana program that runs the real Winterfell verifier—FRI folding, Merkle openings, Fiat–Shamir, constraint evaluation—inside the runtime, and measured it.

It runs. It costs 3.77 million compute units at $k = 4$ and 7.13 million at $k = 64$. Both exceed Solana’s 1.4-million-CU limit for a single transaction, so this needs execution chunked across transactions—which existing work [16] implements with a resumable verifier. The number is consistent with the published Solana STARK-verification study [11], which verifies a Winterfell proof at 1.1M CU for a 4.4KB proof: we spend $3.4\times$ the compute for a $2.7\times$ larger proof.

Then we asked the sharp question: can tuning the parameters bring even the smallest set under the 1.4M limit? We swept the number of FRI queries down to the floor:

queries	CU ($k = 4$)
28	3,768,496
16	2,769,318
8	1,951,812
4	1,567,787

Even at four queries—a security level no one would ship—it is 1.57M CU, still over the cap. The base cost of setup, out-of-domain sampling, and constraint evaluation over a 128-bit field dominates, and the blow-up factor cannot go below eight because the Rescue constraint degree is five. So single-transaction verification is not a tuning problem. It is a *field* problem, and that is now a measurement rather than an argument.

This points cleanly at the fix. Our proof is large, and our verification is expensive, for the same reason: the 128-bit field, where every element is sixteen bytes and every multiplication is several 64-bit operations. A Circle STARK over the 31-bit Mersenne field M31—the modern choice, used in production provers [14] and already demonstrated as an on-chain Solana verifier at $\sim 31k$ CU [15], post-quantum, no ceremony—would shrink both. Porting riverrun’s circuit to a small field is the concrete roadmap, and it is a field change, not a possibility proof.

7.3 What the chain enforces today

Until that port, the deployed on-chain program does not verify the proof itself. It would be dishonest to leave that implicit, so the program makes the trust explicit and bounded rather than hidden. The pool names a *verifier* key; an execution must carry that key’s signature over exactly the tuple being settled, checked through the runtime’s native signature precompile and instruction introspection. This removes the two cheap attacks—an arbitrary signer settling an action, and a watcher front-running a nullifier out of the mempool—because neither can produce the verifier’s signature over their own tuple. It does *not* remove trust: a dishonest verifier can attest to a proof that does not exist. It is a named assumption, not a proof, and the roadmap above is how it goes away. The program also prices a seat in the set (an entry fee, so inflating k costs money rather than nothing) and refuses to settle below a minimum-anonymity floor—anti-Sybil that is honestly “priced, not prevented”, with the stronger one-nullifier-per-verifier construction [10] named as the real fix.

8 What does not hold yet

A paper that only lists its strengths is an advertisement. Here, in one place, is what riverrun does not yet do, so that no reader has to discover it by accident.

- **The circuit is hand-rolled and unaudited.** Mutation testing removes a class of worthless tests, but a passing negative test is necessary, not sufficient, for soundness. An independent review is required before any production claim.

- **It is not formally zero-knowledge.** The underlying prover does not randomize the witness, so the honest claim is “the secret does not appear in the transmitted proof”, checked empirically, not proven.
- **Verification is off-chain today**, behind the named-verifier attestation above. The chain does not yet check the proof; the measured cost and the M31 path (Section 7) are how that changes.
- **Security parameters are example-grade**, roughly 84 bits, not the 128 a deployment needs.
- **“Rootless” provenance is conditional.** The circularity defense holds only while a construction’s funding sources are themselves unattributable; one known exchange entry degrades it, measurably.
- **The ricorso proof is specified, not built**, for the set-size-alignment reason in Section 5.
- **Anti-Sybil is priced, not prevented.** An attacker with money can still buy k ; the fee raises the cost, and the funding graph makes the cheap version of the attack visible to the very ruler of Section 6.

The whole implementation is Rust, MIT-licensed, with 87 passing tests across the cryptographic core, the STARK, the pool, and the on-chain program, and clean static analysis. Every measurement in this paper is reproducible from the repository with a single command.

9 The threat model, stated plainly

It is easy to claim privacy against an attacker you never describe. Here is the one *riverrun* is built against, and the one it is not.

What the adversary can do. They see the entire public ledger, forever: every transaction, every amount, every address, and the full funding graph behind any wallet. They run modern chain-analysis—clustering, timing, graph learning— and they may maintain a tag database that labels exchange and service addresses. They observe the pool’s public transcript $\{R, a, n\}$ for every execution. For the funding-graph results (Section 6) they can additionally observe the provenance class of the *acting* identity—which is cheap, because a fresh withdrawal wallet must be funded from somewhere visible.

What *riverrun* defends. The link between an actor and their action. Given an execution, the adversary should do no better than $1/k_{\text{eff}}$ at naming the member who authored it, where k_{eff} is the effective size of that member’s provenance class, not the advertised set size—and the tools in this paper measure k_{eff} rather than assume it.

What *riverrun* does *not* defend, by design. Amounts and balances— they are public here, and hiding them is a different, composable problem. The value layer of consensus: the ledger records that value moved one way, and no circularity argument changes that; the circularity lives on the attribution layer an analyst reads, not the settlement layer consensus enforces. And, in the deployed program today, the correctness of the membership proof itself, which is delegated to a named verifier (Section 7) until on-chain verification lands.

The reason to write this down is that a privacy claim without a threat model is not falsifiable, and an unfalsifiable claim is not an engineering claim. Every number in this paper is stated against this adversary.

10 Related work

riverrun’s *goal*—hiding which member of a set acted—is anonymous authentication, a mature field. PrivDID [7] states the same session- unlinkability goal; anonymous-credential frameworks with predicate proofs [8, 9] live in the same threat model of a verifier who wants to correlate sessions; PLUME [6] formalizes the deterministic nullifier riverrun uses. The *metric* is Serjantov and Danezis [1]. The *programme* of measuring advertised-versus- true anonymity is established on Ethereum [2, 3, 4, 5]. And on-chain STARK verification on Solana has been done, including with the same Winterfell library [11, 15, 16].

What is narrow and new here is the combination: the payload is behavior rather than value, which is what the target problem asks for; the funding graph is *measured* rather than assumed away, on riverrun’s own constructions as well as on live third-party pools; and that measurement is turned into a user-facing, protocol-agnostic pre-flight defense that, to our knowledge, did not previously exist on Solana.

11 Conclusion

There are two ideas in this paper, and they are stronger together than apart.

The first is a way to hide behavior, not money, on a public chain: commit an intent, execute it unlinkably, and prove the whole thing with a transparent, post-quantum STARK that welds membership, nullifier, and action into one object, at a cost we measured down to the compute unit. It has real, named limits—an unaudited circuit, off-chain verification today, a field that is too big—and a concrete path through each.

The second is a ruler for whether any such system actually works, applied first and hardest to riverrun itself. Every anonymity pool on the chain advertises a number that overstates its privacy, because it counts members and ignores where their money came from, and the funding graph is public. We measured the gap on a live pool—advertised thirty, effective six and a half, worst case one—and turned the ruler into a tool that protects a user before they act, against any pool.

The honest version of a privacy system is one that ships its own strongest attack and reports what it finds. That is what we tried to build. In the words the system takes its name from: *a way a lone last a loved a long the riverrun*—the river returns to its source, and so should every claim, back to the measurement that supports it.

A Reproducing every number in this paper

The system is open-source and MIT-licensed. Every measurement here comes from a command, not a spreadsheet. The whole thing runs from one script, `./demo.sh`, which walks the suites, the mechanism, both adversaries, the effective- k ruler, and—if the Solana toolchain is present—the on-chain program and its tests. Individually:

claim	command
87 tests green	<code>cargo test --workspace</code> (+ the two excluded crates)
one execution’s cost (Table 3)	<code>cargo run --release --example bench</code>
the mechanism, batched round (Table 1)	<code>cargo run --release --example behavior_pool</code>
k_{eff} on riverrun’s designs	<code>riverrun exhibit</code>
k_{eff} of a live pool (Table 2)	<code>riverrun audit <POOL> 30</code>
one wallet’s provenance	<code>riverrun trace <WALLET></code>
your anonymity before you act	<code>riverrun preflight <WALLET></code>
on-chain verification cost	<code>cargo test -p ...stark-verifier --test cu</code>

The live-data commands read only public chain data over a standard RPC endpoint, name no individual in their output, and print their own bounds: a clean result is a floor—“no cheap attribution found”—never a proof of anonymity.

References

- [1] A. Serjantov and G. Danezis. *Towards an Information Theoretic Metric for Anonymity*. Privacy Enhancing Technologies (PET) 2002. Outstanding Paper Award. <https://bib.mixnetworks.org/pdf/serjantov2002towards.pdf>
- [2] N. Wu et al. *Tutela: An Open-Source Tool for Assessing User-Privacy on Ethereum and Tornado Cash*. arXiv:2201.06811. <https://arxiv.org/abs/2201.06811>
- [3] F. Béres, I. A. Seres, A. A. Benczúr, M. Quintyne-Collins. *Blockchain is Watching You: Profiling and Deanonymizing Ethereum Users*. arXiv:2005.14051. <https://arxiv.org/abs/2005.14051>
- [4] *Clustering Deposit and Withdrawal Activity in Tornado Cash*. arXiv:2510.09433 (2025). <https://arxiv.org/abs/2510.09433>
- [5] H. Du, Z. Che, M. Shen, L. Zhu, J. Hu. *Breaking the Anonymity of Ethereum Mixing Services Using Graph Feature Learning*. IEEE TIFS, 2024. <https://doi.org/10.1109/TIFS.2023.3326984>
- [6] A. Gupta and K. Gurkan. *PLUME: An ECDSA Nullifier Scheme for Unique Pseudonymity within Zero Knowledge Proofs*. IACR ePrint 2022/1255. <https://eprint.iacr.org/2022/1255>
- [7] *Toward Verifiable Privacy in Decentralized Identity: A Formal Framework for Minimal Disclosure and Unlinkability (PrivDID)*. IACR ePrint 2026/127. <https://eprint.iacr.org/2026/127>
- [8] *Formalizing Privacy of Anonymous Credentials: A Provably Secure Framework with Predicate Proofs*. IACR ePrint 2026/1373. <https://eprint.iacr.org/2026/1373>
- [9] *Re2creds: Reusable Anonymous Credentials*. IACR ePrint 2026/119. <https://eprint.iacr.org/2026/119>
- [10] *Anonymous Self-Credentials and their Application to Single-Sign-On*. IACR ePrint 2025/618. <https://eprint.iacr.org/2025/618>
- [11] J. Yano. *Full L1 On-Chain ZK-STARK+PQC Verification on Solana: A Measurement Study*. IACR ePrint 2025/1741. <https://eprint.iacr.org/2025/1741>
- [12] A. Szepieniec, T. Ashur, S. Dhooghe. *Rescue-Prime: a Standard Specification (SoK)*. IACR ePrint 2020/1143. <https://eprint.iacr.org/2020/1143>
- [13] Facebook/Meta. *Winterfell: a STARK prover and verifier*. <https://github.com/facebook/winterfell>
- [14] StarkWare. *Stwo: a production Circle STARK prover over M31*. <https://github.com/starkware-libs/stwo>

- [15] *Murkl: a Circle STARK verifier as a Solana CPI target.* <https://github.com/exidz/murkl>
- [16] Wiener Labs. *Mosaic: a trait-based on-chain verifier library for Solana (chunked FRI-STARK).* <https://github.com/wienerlabs/mosaic>